Aufgabe 1: Kritische Abschnitte

Synchronisation & Kritische Abschnitte – Formelsammlung & Übersicht

Kritischer Abschnitt (Critical Section): Programmabschnitt, in dem auf gemeinsame Ressourcen (Variablen, Dateien, Hardware) zugegriffen wird.
Race Condition (Wettlaufsituation): Fehlerhafter Zustand, bei dem das Endergebnis unkontrolliert von der zeitlichen Ausführungsreihenfolge (Scheduling) der Threads abhängt.
Vier Bedingungen für korrekten wechselseitigen Ausschluss (Tanenbaum):

1. **Mutual Exclusion:** Niemals dürfen sich zwei Prozesse gleichzeitig im kritischen Abschnitt befinden.
2. **No Assumptions:** Keine Annahmen über Ausführungsgeschwindigkeiten oder Prozessoranzahl.
3. **Progress:** Kein Prozess außerhalb des kritischen Abschnitts darf andere blockieren.
4. **Bounded Waiting (Fairness):** Kein Prozess darf unendlich lange auf Einlass warten müssen (kein Verhungern / Starvation).

Aufgabe 1: C#-Threads & Gemeinsame Variable (Shared ID)

Interaktive Simulation von Race Conditions in C#-Threads

Simulator In

der Vorlesung haben wir folgendes Multithreading-Beispiel behandelt:

```
using System;
using System.Threading;

namespace Threads
{
    class Program
    {
        private const int NumberOfThreads = 10;

        static void StaticMethod(object threadId)
        {
            Console.WriteLine($"Hello World. Greetings from thread {threadId}");
        }

        private static void Main(string[] args)
        {
            for (var i = 0; i < NumberOfThreads; i++)
            {
                Console.WriteLine($"Main here. Creating thread {i}");
                var newThread = new Thread(StaticMethod);
                newThread.Start(i);
            }
        }
    }
}
```

a) Wie müssten Sie das Programm erweitern, sodass ein kritischer Abschnitt entsteht und in der Programmausgabe zumindest ab und zu der Fehler (Race Condition) deutlich wird?

b) Wie lässt sich dieser kritische Abschnitt in C# absichern, um die Race Condition vollständig zu verhindern?

Aufgabe 2: Lost Update beim Bankkonto (Read-Modify-Write)

Interaktive Demonstration des Lost-Update-Phänomens

Simulator ↗

Zwei Threads T_1 und T_2 greifen nebenläufig auf dasselbe Bankkonto mit dem Anfangssaldo **100 €** zu. T_1 hebt **50 €** ab, T_2 hebt **30 €** ab. Die Abhebung besteht aus drei internen Einzelschritten:

1. **Read:** Saldo aus dem Hauptspeicher in das thread-lokale Register R laden (`R = saldo`)
2. **Modify:** Betrag im Register subtrahieren (`R = R - betrag`)
3. **Write:** Neuen Wert aus dem Register in den Hauptspeicher zurückschreiben (`saldo = R`)

Betrachten Sie folgendes unglückliches Interleaving:

Schritt	Aktion	R_1	R_2	Saldo im Speicher
0	Start	–	–	100 €
1	T_1: `R1 = saldo`	100 €	–	100 €
2	T_2: `R2 = saldo`	100 €	100 €	100 €
3	T_1: `R1 = R1 - 50`	50 €	100 €	100 €
4	T_1: `saldo = R1`	50 €	100 €	
5	T_2: `R2 = R2 - 30`	50 €	70 €	50 €
6	T_2: `saldo = R2`	50 €	70 €	

c) Welcher Endsaldo steht nach Schritt 6 im Speicher, und welcher Betrag hätte bei korrekter Ausführung im Speicher stehen müssen?

Aufgabe 3: Shared Counter & Assembler-Instruktionen

Interaktives Interleaving auf Maschinenbefehlsebene

Simulator ↗

Eine globale Zählervariable `count` wird von Thread A inkrementiert (`count++`) und von Thread B dekrementiert (`count--`). Auf Assemblerebene besteht der Inkrement-Befehl aus:

- `MOV R1, [count]` (Laden)
- `ADD R1, 1` (Inkrementieren)
- `MOV [count], R1` (Speichern)

Analog dekrementiert Thread B über Register `R2`. Der Anfangswert von `count` ist **5**.

d) Geben Sie alle möglichen Endwerte von `count` nach je einer vollständigen Ausführung beider Operationen an, wenn ein beliebiges Interleaving auftreten kann.

Aufgabe 4: Vier Kriterien für wechselseitigen Ausschluss

Tanenbaum-Kriterien für Synchronisationsprimitive

Simulator ↗

Um kritische Abschnitte korrekt abzusichern, müssen Synchronisationsverfahren strenge Kriterien erfüllen.

e) Nennen Sie die vier Bedingungen nach Tanenbaum/Dijkstra für eine korrekte Lösung des Problems des kritischen Abschnitts.

f) Was versteht man unter *Busy Waiting* (aktivem Warten) und warum ist es auf Uniprozessorsystemen besonders schädlich?

Aufgabe 5: Petersons Algorithmus (Zwei-Prozesse-Synchronisation)

Interaktiver Peterson-Simulator im Web **Simulator** ↗

Petersons Algorithmus garantiert wechselseitigen Ausschluss für zwei Prozesse P_0 und P_1 im User-Space ohne Hardware-Unterstützung:

Prozess P_0	**Prozess P_1**
`flag[0] = true;`	`flag[1] = true;`
`turn = 1;`	`turn = 0;`
`while (flag[1] && turn == 1); // warten`	`while (flag[0] && turn == 0); // warten`
`// Kritischer Abschnitt`	`// Kritischer Abschnitt`
`flag[0] = false;`	`flag[1] = false;`

g) Welche Rolle spielen das Array `flag[]` und die Variable `turn`?

h) Warum funktioniert Petersons Algorithmus auf modernen Mehrkern-CPUs nicht mehr ohne Weiteres ohne *Memory Barriers* (Speicherbarrieren)?